Paper VIII
Third Lean Verification Block

Packet Operator Algorithms, Exact Compilation,
and GPU Realization

Komorebi P. September 6, 2026

Scope and result

This supplement continues the strengthened manuscript dated August 14, 2026. It records checked mathematics without replacing the original manuscript. The original source package, PDF, TeX, bibliography, theorem register, both previous supplements, and all 11 upstream Lean modules are preserved unchanged.

Nine additional named manuscript results are covered. The cumulative status is **40 of 56 manuscript-proved items**, with 16 still not fully formalized. Four classical inputs are tracked separately. This block adds **68 theorem declarations in seven modules**, giving **197 declarations across 18 modules**. Supporting lemmas are included in these declaration counts.

New area	Source items	Declarations
Product-chain error, repeated recompression, packet error propagation	5.8, 5.9, 8.3	13
Constructed low-rank product chains	5.2	5
Exact and approximate packet Krylov spaces	7.2, 7.3	11
Budget convolution, quantized tree compilation, discrete precision, stored arrays	8.7, 9.8, 9.9	39

All 18 modules passed a fresh cumulative run, with 197 separate theorem audits and 12 separate construction audits. Both deliberately invalid controls were rejected. The existing verifier's 17 regression tests also passed.

New executable tests compare the cached programs with independent exhaustive enumeration using exact rational costs. They cover 384 tree instances and 288 precision instances. The earlier 600-case compiler and cached-update suite was also rerun successfully. These are executable regressions, not additional universal theorems.

Boundary. The compiler solves a finite additive model with integer error weights. The work bound is a declared table-operation model. No universal discovery algorithm, CUDA correctness theorem, GPU timing result, or new numerical benchmark is claimed.

1. Low-rank factors through a product chain

Source item 5.2

Let

$$A_j = D_j + U_j V_j^*, \qquad U_j, V_j \in \mathbb{C}^{n \times r_j},$$

with all D_j diagonal in one packet layout. Set

$$P_m = A_m \cdots A_1, \qquad D^{(m)} = D_m \cdots D_1.$$

The formalization constructs thin matrices L_m, R_m such that

$$\boxed{P_m = D^{(m)} + L_m R_m^*.}$$

Their column index type is a recursively constructed disjoint sum. Its cardinality is proved to equal $\sum_{j=1}^{m} r_j$.

Construction. Suppose $P_{m-1} = D^{(m-1)} + L_{m-1} R_{m-1}^*$. The new factors are

$$L_m = \begin{bmatrix} D_m L_{m-1} + U_m (V_m^* L_{m-1}) & U_m \end{bmatrix},$$
$$R_m = \begin{bmatrix} R_{m-1} & (D^{(m-1)})^* V_m \end{bmatrix}.$$

Substitution into the already checked two-factor product identity proves exactness. This is not a rank-only existence argument. The recursively defined factors are themselves audited.

Applying the ordinary matrix-rank inequality gives

$$\boxed{\operatorname{rank}(P_m - D^{(m)}) \le \sum_{j=1}^{m} r_j.}$$

The bound concerns the actual correction matrix, not just the number of allocated columns. The proof also shows that $D^{(m)}$ stays packet diagonal. The empty chain has identity base, empty column type, and zero correction. Individual $r_j = 0$ are permitted.

What is not completed here. This is an unrecompressed representation. Optimal small-core singular-value recompression, item 5.6, remains separate. The construction can accumulate redundant columns and is not a claim of minimal factor width.

Actual noncommutative products

The error module defines the left-accumulated product recursively, with $P_0 = I$ and $P_{j+1} = A_{j+1} P_j$. For a perturbed sequence $\widehat{A}_j$, it derives the discrepancy recursion

$$\widehat{P}_{j+1} - P_{j+1} = \widehat{A}_{j+1}(\widehat{P}_j - P_j) + (\widehat{A}_{j+1} - A_{j+1}) P_j.$$

The identity follows from the actual products. It is not supplied as an assumption to a norm estimate, and it does not commute the operators. Lean uses zero-based indices internally, so index j in source code corresponds to manuscript factor $j + 1$.

2. Error propagation and recompression

Source items 5.8, 5.9, and 8.3

For $\widehat{A}_j = A_j + E_j$, the checked product-chain bound is

$$\boxed{\left\|\widehat{A}_m \cdots \widehat{A}_1 - A_m \cdots A_1\right\| \le \sum_{j=1}^{m} \left(\prod_{i=j+1}^{m} \left\|\widehat{A}_i\right\|\right) \|E_j\| \left(\prod_{i=1}^{j-1} \|A_i\|\right).}$$

Empty products are one. The proof first solves a scalar gain recurrence as a finite sum, then bounds the actual discrepancy recursion by it. A version with certified upper bounds $\delta_j \ge \|E_j\|$ is included. The product-norm argument uses a norm-one identity, as in operators on a nonzero normed space.

Post-product recompression is different. Define the implemented iterates by

$$\widetilde{P}_0 = I, \qquad \widetilde{P}_j = A_j \widetilde{P}_{j-1} + E_j.$$

Here E_j is a residual introduced after multiplication. Lean derives

$$\widetilde{P}_j - P_j = A_j(\widetilde{P}_{j-1} - P_{j-1}) + E_j$$

and proves

$$\boxed{\left\|\widetilde{P}_m - P_m\right\| \le \sum_{j=1}^{m} \left(\prod_{i=j+1}^{m} \|A_i\|\right) \delta_j, \quad \|E_j\| \le \delta_j.}$$

This theorem works in a normed ring without the extra norm-one condition. It includes the empty iteration. There is no exact-prefix factor in the local residual term, because the residual is already measured at the post-product output.

Packetwise mixed precision. At each stage let

$$A_j = \sum_e A_{j,e}, \qquad \widehat{A}_j = \sum_e (A_{j,e} + \Delta_{j,e}), \qquad \Delta_{j,e} = \Delta_{j,e} P_{j,a(e)}.$$

The previous packet certificate supplies the uniform operator bound

$$\left\|\widehat{A}_j - A_j\right\| \le \delta_j := \sum_e \|\Delta_{j,e}\| \left\|P_{j,a(e)}\right\|.$$

The new theorem substitutes these δ_j into the actual product-chain estimate. The resulting vector error is at most the operator certificate times $\|x\|$. A vector-specific packet bound is not silently reused as an operator norm.

These are deterministic implications from stated residual bounds. They do not establish the rounding behavior of a floating-point kernel. The floating-point matrix-product model remains one of the separately tracked classical inputs.

3. Exact and approximate packet Krylov spaces

Source items 7.2 and 7.3

Write

$$\mathcal{K}_m(A, v) = \operatorname{span}\{v, Av, \ldots, A^{m-1}v\}.$$

For a complete finite projector family P_i, suppose A commutes with every P_i. Let $A_i = P_i A P_i$. Lean proves the actual orbit identity

$$A^j v = \sum_i A_i^j P_i v$$

and therefore

$$\boxed{\mathcal{K}_m(A, v) \subseteq \bigoplus_i \mathcal{K}_m(A_i, P_i v).}$$

The case $j = 0$ uses $\sum_i P_i = I$. The zeroth local power is the ambient identity, applied to $P_i v$, rather than an incorrectly substituted local projector power.

The direct sum is justified. Each local space is proved to lie in the actual range of P_i. Projecting a finite sum of local vectors selects its ith summand. Consequently a zero sum of local vectors has every summand zero. The proof does not assert equality between the global Krylov space and the full sum of local Krylov spaces.

For approximate invariance let $A = D + K$, where D is packet diagonal, and set

$$M = \max\{\|A\|, \|D\|\}.$$

For every positive integer j, the operator estimate is

$$\boxed{\left\|A^j - D^j\right\| \leq j M^{j-1} \|K\|.}$$

The proof is noncommutative and uses actual powers. Its positive-index version works even in a trivial normed ring. No division by M is used, so $M = 0$ causes no exception.

Define the comparison space

$$\mathcal{L}_{j+1} = \bigoplus_i \mathcal{K}_{j+1}(P_i D P_i, P_i v).$$

Packet diagonality is first shown to imply commutation. The exact theorem then proves $D^j v \in \mathcal{L}_{j+1}$. Using that vector as a witness in the infimum distance gives

$$\boxed{\operatorname{dist}(A^j v, \mathcal{L}_{j+1}) \leq j M^{j-1} \|K\| \|v\|.}$$

This statement uses Lean's actual metric infimum distance to the subspace. It does not assume that the comparison vector lies there. The block-Jacobi condition bound and the preconditioned conjugate-gradient corollary are not completed by these Krylov results.

4. Integer error budgets and discrete precision

Source item 8.7 and common minimum machinery

The finite budget programs use integer weights and exact ordered-field costs. They apply to rational costs for execution and to real costs for mathematical statements. Unreachable states are represented by `none`, not by a large finite sentinel.

For a predicate S on costs, the common minimum specification means:

$$\begin{array}{ll} \texttt{none}: & \text{no cost satisfies } S, \\ \texttt{some}(v): & S(v) \text{ and } v \le z \text{ whenever } S(z). \end{array}$$

Both attainment and minimality are included. Optional minimum and optional addition are proved to implement union and independent sum of feasible sets.

For exact-weight tables F, G, min-plus convolution is

$$(F \star G)(b) = \min_{0 \le k \le b} \{F(k) + G(b-k)\},$$

with an infeasible sum whenever either operand is infeasible. Its correctness theorem identifies every finite answer with an actual feasible pair of component weights and costs. Repeated convolution handles arbitrary finite lists of components.

Independent precision assignments. For every edge, a finite list contains its permitted pairs (w, c) of quantized error and cost. The legal assignment type chooses exactly one listed pair per edge. It is defined independently of the dynamic program. Costs and weights are then summed over that assignment.

If D is the already-solved tail table, the next edge has exact-weight recurrence

$$F(b) = \min_{(w,c) \text{ allowed},\ w \le b} \{D(b-w) + c\}.$$

The final answer is $\min_{0 \le b \le B} F(b)$. Lean proves that this is the exact minimum over legal assignments with total error at most B, or `none` exactly when no such assignment exists.

The subtraction guard matters. Natural subtraction truncates at zero. Without the condition $w \le b$, an option of weight one could be accepted at budget zero. The implemented program checks the condition before subtraction. Dedicated regressions also cover a zero-weight option, an empty option set, and the empty collection of edges. The last case has cost and error zero.

This result does not prove that an unquantized physical error model is accurately represented by integer weights. The formal optimum concerns the declared quantized model. Clipped continuous allocation, item 8.5, remains separate.

5. The quantized packet-tree compiler

Source item 9.8

The new program uses the previous block's actual finite `CandidateTree` and independent legal `Cut` type. An internal node has any nonempty finite family of children. A cut either keeps its node or chooses a legal cut in every child. A terminal node can only be kept.

Keeping node v costs κ_v and incurs integer error w_v. Splitting costs overhead σ_v in addition to the selected child cuts. The exact-weight recurrence is

$$F_v(b) = \min \begin{cases} \kappa_v & \text{if } b = w_v, \\ \sigma_v + \min\limits_{\sum_i b_i = b} \sum\limits_i F_{v_i}(b_i) & \text{if } v \text{ is internal.} \end{cases}$$

Nonexistent alternatives are infeasible. The child sum is implemented by the proved finite min-plus convolution, not by assuming an optimal allocation.

Exact semantics. For every tree and every integer weight b, Lean proves that $F_v(b)$ is a minimum of costs of independent legal cuts of exact weight b. Thus each finite answer has an attaining cut and is a lower bound for every other legal cut at that weight. Infeasibility is equivalent to the absence of such a cut.

The root scan

$$\boxed{\min_{0 \le b \le B} F_{\mathrm{root}}(b)}$$

is therefore the exact budget-constrained optimum. Increasing the budget cannot increase a finite optimum. The theorem retains feasibility hypotheses where necessary rather than treating `none` as an ordinary real cost.

Why the budget program adds something. Consider the three admissible cost/error pairs

$$(0, 4), \qquad (3, 2), \qquad (4, 0).$$

At budget two the answer is cost three. The middle pair cannot minimize cost plus λ times error: comparison with the first pair would require $\lambda \ge 3/2$, while comparison with the third would require $\lambda \le 1/2$. The executable suite checks the budget answer on an actual unary candidate tree with these cuts. It is not obtained merely by varying the previous Lagrangian compiler's multiplier.

Output boundary. This budget implementation returns the minimum cost or `none`. Attainment is proved, but executable backtracking metadata for a selected cut is not yet returned. Universal packet discovery, overlapping decompositions, nonadditive cache costs, and arbitrary device scheduling remain outside the theorem.

6. Cached arrays and a work certificate

Source item 9.9 and implemented forms of 8.7 and 9.8

A recursive value equation alone need not be an efficient program. Repeated calls can recompute the same subtree table. The new implementation therefore stores each table as an array-backed `Vector` with exactly $B + 1$ optional entries.

Each child table is evaluated and cached before the parent merges it. A merge reads two existing tables and tabulates their convolution. The discrete precision program likewise computes its tail table once before filling the next edge's table. The final scan reads the stored root or final-edge table.

Entrywise implementation theorem. For every $b \le B$, Lean proves

$$\mathrm{lookup}(\mathrm{storedTree}(B, t), b) = F_t(b).$$

There is a parallel theorem for stored discrete precision. The proof establishes first that an output entry at b depends only on input entries at most b, then proceeds through cached convolution and the tree recursion. Consequently the optimality, attainment, and infeasibility specification applies to the actual stored result.

Counted candidates. One full convolution table examines indices

$$(b, k), \qquad 0 \le b \le B, \quad 0 \le k \le b.$$

These form a concrete finite dependent-sum type. Its cardinality is proved to be

$$\sum_{b=0}^{B} (b + 1) \le (B + 1)^2.$$

For a nonempty finite tree, the number of child merges plus one equals the node count N. This identity is proved for arbitrary finite branching.

Let $W_B(t)$ be the declared unit-cost count of table-cell constructions and tested min-plus candidates. The model charges the empty child-combination table as well as the output cells and merge candidates. Lean proves

$$W_B(t) \le 2(B + 1)N + \big((B + 1) + (B + 1)^2\big)(N - 1)$$

and hence

$$\boxed{W_B(t) \le N\big(3(B + 1) + (B + 1)^2\big).}$$

The root minimum scan adds a linear pass. Thus the declared tabulation model is pseudo-polynomial in N and the numeric budget B.

Scope of this certificate. These are table and candidate counts, not a measured execution trace or a theorem about every instruction emitted by the compiler. Rational bit complexity, allocation overhead, memory traffic, and hardware timing are not included. The certificate is not polynomial in the binary input length of B.

7. Verification, executable checks, and remaining work

Release evidence and precise completion boundary

The fresh cumulative run used **Lean 4.34.0-rc2** and pinned Mathlib revision

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`.

All 18 modules returned exit code zero with warnings treated as errors. Every one of the 197 theorem audits used only `propext`, `Classical.choice`, and `Quot.sound`, or subsets thereof. Twelve constructions were separately audited. The active proof files contain no admitted proofs or added axiom declarations. The deliberately false equality and admitted-proof control were both rejected.

Exact-rational executable suite. Independent enumeration, rather than the DP recurrence, produces the candidate costs used for comparison.

New test category	Count
Tree instances with unary, binary, ternary, and mixed branching	384
Exact-weight table comparisons for trees	4,992
Budget-minimum comparisons for trees	4,992
Independently enumerated legal cuts	19,152
Additional functional-specification comparisons	1,440
Discrete precision instances	288
Exact-weight / budget comparisons for precision	3,744 each
Independently enumerated legal assignments	1,453
Explicit boundary regressions	5

All passed. The preceding block's 600 compiler cases, 85,320 enumerated pruning scores, and 600 cached-update comparisons were also rerun and passed. These counts describe executable tests, not additional theorem declarations.

Failures retained. Eleven of 18 captured development compilation checks failed before correction. Their source snapshots and diagnostics are retained separately from the successful final run. They principally concern type elaboration, module visibility, finite indices, and simplification. An unsupported interactive runtime-recovery request failed before assembly started. The later noninteractive restoration succeeded.

Sixteen manuscript-proved items remain: 4.1, 4.3–4.7, 5.6, 6.5–6.8, 7.4, 7.6, 8.5, 9.2, and 10.3. These concern structured spectral and tensor primitives, optimal recompression, graph scheduling and inverse support, preconditioning, clipped allocation, and block-discovery certificates. Four classical inputs are still separate.

No GPU timings, original CPU numerical suite, full Papers I–VII rebuild, or independent recheck of imported Mathlib proofs was performed in this block. The JSON verification report, execution reports, preservation report, source ledger, logs, and scripts provide the auditable release record.